NewTech

Academy of Technology

Order System

Smart Restaurant

A complete JavaScript exercise combining variables, arrays, objects, functions, conditions, loops, and array methods.

Duration	Level	Topic	Parts
2 Hours	Intermediate	JavaScript	11 + 4 Bonus

OBJECTIVE

Build a complete JavaScript mini-system that simulates a restaurant ordering process. The exercise combines **variables, arrays, objects, functions, if/else conditions, loops, array methods, object manipulation, basic validation,** and console output.

TIME PLAN

Part	Topic	Time
1	Data Setup	15 min
2	Menu Functions	25 min
3	Order Logic	35 min
4	Payment and Discount	25 min
5	Receipt and Bonuses	20 min

15 min

Create the Menu Array

Create an array called menu. It should contain at least **8 items**. Each menu item is an object with the following properties:

✓ id — unique number

- ✓ name — item name
- ✓ category — Food / Drink / Dessert
- ✓ price — number in NIS
- ✓ available — true or false

Example item object:

```
const menu = [
{
id: 1,
name: "Burger",
category: "Food",
price: 35,
available: true
},
// ... add at least 7 more items
];
```

Create a Customer Object

Create an object called customer with:

- ✓ name — customer's name
- ✓ budget — amount of money in NIS (example: 120)
- ✓ isStudent — true or false

Create an Order Array

Create an array called order that contains item IDs from the menu.

```
const order = [1, 3, 5]; // IDs of items the customer wants
```

25 min

Function: displayMenu()

Create a function that prints all menu items in a clear format. For each item, print:

- ✓ ID

✓ Name

✓ Category

✓ Price (in NIS)

✓ Availability (Available / Not Available)

Expected output:

```
// Example output:
1 - Burger - Food - 35 NIS - Available
2 - Pizza - Food - 50 NIS - Available
3 - Orange Juice - Drink - 12 NIS - Not Available
```

Requirement: Use a loop — you may use for, for...of, or forEach.

Function: getAvailableItems()

Create a function that returns only the items where available is true.

Requirement: Use filter().

Function: findItemById(id)

Create a function that searches the menu and returns the item matching the given id. If no item is found, return a message: Item not found.

Requirement: Use find().

35 min

Function: getOrderItems()

Convert the order array (list of IDs) into an array of full item objects from the menu.

```
// Input:
const order = [1, 3, 5];

// Output:
// [burgerObject, colaObject, iceCreamObject]
```

Requirement: Use map() and find().

Function: validateOrder()

Check two things for every item in the order:

✓ The item exists in the menu
✓ The item is available (available === true)

```
// If item doesn't exist:
console.log("This item does not exist.");

// If item exists but unavailable:
console.log("Sorry, this item is currently not available.");

// Return true if all valid, false otherwise
```

25 min

Function: calculateTotal()

Calculate the total price of all ordered items.

Requirement: Use reduce().

```
// Example:
Burger: 35
Cola: 8
Ice Cream: 15

Total: 58
```

Function: applyDiscount()

Apply discount rules to the total. Use only the **biggest** applicable discount:

✓ Customer is a student → 10% discount
✓ Total is more than 100 NIS → 15% discount
✓ Total is more than 150 NIS → 20% discount
✓ No rule applies → no discount

The function should return an **object** with:

```
{
originalTotal: 120,
discountPercentage: 15,
discountAmount: 18,
finalTotal: 102,
}
```

Function: canCustomerPay()

Check if the customer's budget covers the final total.

- ✓ If customer has enough money → return true
- ✓ If not → return false

20 min **Function: printReceipt()**

Print a complete receipt to the console. The receipt must include:

- ✓ Restaurant name
- ✓ Customer name
- ✓ Ordered items with prices
- ✓ Original total
- ✓ Discount percentage
- ✓ Discount amount
- ✓ Final total
- ✓ Customer budget
- ✓ Payment status

Expected output:

========== RECEIPT ========== Restaurant: JavaScript Burger House Customer: Ahmad Items: - Burger: 35 NIS - Cola: 8 NIS - Ice Cream: 15 NIS Original Total: 58 NIS Discount: 10% Discount Amount: 5.8 NIS Final Total: 52.2 NIS Customer Budget: 100 NIS Payment Status: Paid Successfully ==============================
Note: If the customer cannot pay, the Payment Status should be: Not Enough Money

At the end of your file, connect all functions together in this order:

1	Display the full menu
2	Display only available items
3	Convert order IDs into full item objects
4	Validate the order
5a	If valid: calculate total → apply discount → check budget → print receipt
5b	If invalid: stop the order and print a clear error message

Complete the main exercise first, then try these for extra challenge.

Create a function called **countItemsByCategory()** that counts how many ordered items belong to each category.

Requirement: Use reduce().

```
// Expected output:
{
Food: 2,
Drink: 1,
Dessert: 1
}
```

Create a function called **getMostExpensiveItem()** that returns the most expensive item in the customer's order.

```
// Expected output:
Most expensive item: Pizza - 50 NIS
```

Change the order structure to include quantities, then update your total calculation:

```
// New order structure:
const order = [
{ id: 1, quantity: 2 },
{ id: 3, quantity: 1 },
{ id: 5, quantity: 3 },
];

// Expected total calculation:
Burger x2 = 70
```

Cola x1 = 8
Ice Cream x3 = 45

Total = 123

Add a variable called **orderStatus** and update it based on the result:

✓ Order invalid → Rejected
✓ Order valid, customer cannot pay → Rejected
✓ Order valid, customer can pay → Paid

`let orderStatus = "Pending";`
 	
`// Update to one of:`
`"Pending" | "Approved" | "Rejected" | "Paid"`